

TUTORIAL SHEET – LINUX & SHELL SCRIPTING

Total Questions: 40

SECTION A – MULTIPLE CHOICE QUESTIONS (20 MCQs)

Choose **one** correct answer.

1. Which component of Linux is responsible for managing processes, memory, and hardware?

- A. Bash
- B. Kernel
- C. Shell
- D. Desktop Environment

2. Which of the following is NOT a Linux distribution?

- A. Ubuntu
- B. Fedora
- C. Windows 10
- D. Debian

3. What does the command `cd ~` do?

- A. Changes to root directory
- B. Lists current directory
- C. Moves to the user's home directory
- D. Clears the terminal

4. Which command shows a detailed listing with permissions, owner, and size?

- A. `ls`
- B. `ls -l`
- C. `ls -a`
- D. `dir`

5. Which path type always begins with a slash `/`?

- A. Relative path
- B. Temporary path
- C. Absolute path
- D. Symbolic path

6. Which command displays the first 10 lines of a file?

- A. `tail`
- B. `head`
- C. `cat`
- D. `less`

7. Which command allows you to edit a file inside the terminal using a simple interface?

- A. `nano`
- B. `chrome`
- C. `gcc`
- D. `cal`

8. The shebang line always appears:

- A. At the bottom of a script
- B. On any line
- C. At the top of a script
- D. Only inside loops

9. Which command makes a script executable?

- A. `chmod 777 script.sh`
- B. `chmod +x script.sh`

- C. `sudo script.sh`
- D. `sh script.sh`

10. What happens when you run a script with `./script.sh`?

- A. The kernel compiles it
- B. Bash executes it directly
- C. It opens in nano
- D. It creates a new user

11. In Bash, variables are assigned using:

- A. `name := value`
- B. `let name = value`
- C. `name=value`
- D. `$name=value`

12. An environment variable is made using:

- A. `env var=value`
- B. `global var=value`
- C. `export var=value`
- D. `sys var=value`

13. What is the purpose of `$@`?

- A. Removes all variables
- B. Returns number of parameters
- C. Expands to all script arguments as separate words
- D. Displays last command run

14. What is the output of: `echo $(echo hello)` ?

- A. `hello`
- B. `$(hello)`
- C. `$(echo hello)`
- D. `hello hello`

15. Which loop continues until a condition becomes true?

- A. `for`
- B. `while`
- C. `until`
- D. `repeat`

16. The `break` statement inside a loop:

- A. Restarts the loop
- B. Skips the loop condition
- C. Exits the loop
- D. Exits the whole script

17. Which function definition is valid in Bash?

- A. `function check_cpu() {}`
- B. `def check_cpu() {}`
- C. `check_cpu() { commands; }`
- D. `fn check_cpu {}`

18. `set -x` is used for:

- A. Enabling debugging and tracing
- B. Disabling user inputs
- C. Creating new variables
- D. Resetting the terminal

19. In the `/proc/stat` file, CPU idle time is usually represented by which field?

- A. `user`
- B. `idle`
- C. `interrupt`
- D. `steal`

20. In the System Health Monitor project, the loop that checks usage every 5 seconds should run for:

- A. Forever
- B. 30 seconds

- C. 5 minutes
- D. Until the disk is full

SECTION B – STRUCTURED QUESTIONS (20 QUESTIONS)

Write clear, complete answers. Diagrams, examples, and short scripts are allowed.

1. Explain the difference between Linux, GNU, and the Linux kernel.
2. Describe the historical origin of Linux. Include the role of Minix and GNU.
3. Using examples, explain the differences between absolute and relative paths.
4. Write the commands to:
 - a) Create a directory named `projects`
 - b) Go inside it
 - c) Create a file called `notes.txt`
5. Explain the difference between `cat`, `less`, and `head`. Provide the best use-case for each.
6. Describe the step-by-step process of executing a Bash script from creation to runtime.
7. Create a Bash script that prints the username and the current date using:
 - variables
 - command substitution
8. Explain the difference between local variables and environment variables with examples.
9. Write a script that asks the user for their name and then prints:
"Hello, <name>, welcome to Linux scripting."
10. What are positional parameters? Explain the roles of `$1`, `$*`, and `$#` with examples.
11. Write a `for` loop that prints all `.txt` files in the current directory.
12. Write a `while` loop that counts from 1 to 10 but skips number 5 using `continue`.

13. Write an `until` loop that waits for a file named `done.txt` to appear, checking every 2 seconds.

14. Define a function in Bash named `greet()` that takes one argument and prints a greeting message.

15. Explain how debugging with `set -x` works. Show an example where debugging helps find a mistake.

16. What is CPU idle time? Explain how `top` or `/proc/stat` can be used to calculate CPU usage.

17. Write a function `check_memory()` that prints total and used memory using `free -m`.

18. Describe how to extract CPU usage using either:

- `top -bn1`
or
- `/proc/stat`

Show a sample `awk` command.

19. In the System Health Monitor project, explain how the script automatically stops after 30 seconds. How is timing implemented?

20. Write a simplified version of the System Health Monitor script including:

- user input for log file
- CPU, Memory, Disk functions
- while loop running every 5 seconds
- log output appended to the file

(A full script is expected.)